PROJECT-BASED LEARNING

PHASE-I EVALUATION

GRAPH BASED FRIEND AND OPPORTUNITY RECOMMENDATION SYSTEM

Team ID: DSCPP-III-2026-T209 — Team Name: Tech-mates

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T209
Team Name: Tech-mates
Semester: 3rd
Section: F,G,DS2,ML1
Domain: Data Structures & Algorithms/C++

Team Members

1. Aaradhya – 2510010209 (Team Lead)
2. Deepak Singh Rana – 2510380196
3. Vanshika Nagrey – 2510370219
4. Akshra Raju – 25022036

Mentor

Mr. Piyush Agarwal • **Interactions completed:** 4

Problem Statement & Motivation

Problem Statement

Existing platforms mainly connect users through existing friendships, without effectively leveraging shared interests, skills, courses, workshops, projects, seminars, or competitions to surface relevant people and opportunities.

Motivation

- Common interests/skills/courses often go unutilized
- Students benefit from peers sharing workshops, competitions, seminars
- Demonstrates DSA (graph traversal, hashing, priority queues) on a real problem

Target Users / Stakeholders

- Students / platform users seeking relevant connections
- Institutions promoting collaboration and skill-sharing
- Benefit: better-matched friends, opportunities, learning experiences

Objectives

Project Objectives

1. Develop a C++-based friend and opportunity recommendation system.
2. Represent social relationships using an undirected graph with adjacency lists (linked lists).
3. Implement BFS and DFS for connection-distance exploration and connectivity checking.
4. Use hashing for fast user/attribute lookup and a priority queue for ranking.
5. Persist users, friendships, and activities using Neo4j; process everything in C++.

Key Objective: Generate ranked friend and opportunity recommendations based on mutual connections and shared attributes (skills, courses, workshops, projects).

Proposed Solution

Proposed Approach

1. Load user, friendship & activity records from Neo4j
2. Build undirected graph (users = nodes, friendships = edges)
3. Traverse graph using BFS (distance/level) and DFS (deep connectivity)
4. Compute compatibility score from mutual friends & shared attributes
5. Rank candidates using a priority queue and output recommendations

Key Features

- Custom linked-list based adjacency list
- Mutual friend & common-interest detection
- BFS-based level-wise connection discovery
- DFS-based deep connectivity checking
- Priority-queue based ranked recommendations

The system extends the classic "friends-of-friends" graph model by also factoring in shared skills, courses, seminars, workshops, competitions, certifications, and projects.

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures & OOPs with C++	Graph (adjacency list via custom linked lists), Queue (BFS), Stack/Recursion (DFS), Hashing (unordered_map), Priority Queue (ranking), Classes for User/Activity/Graph, STL	Recommendation Engine, Graph Module, User & Friend Management
DBMS (Neo4j Graph DB)	Nodes/Relationships for Users, Friendships, Activities, Interests; storage kept separate from processing logic	Data Persistence Layer

Requirement: Demonstrate meaningful integration of both subjects in **one** project.

System Architecture / Workflow



Major Components

- User Management & Friend Management
- Activity/Experience Manager
- Recommendation Engine (BFS/DFS + scoring)

Data / Control Flow

- Records loaded from Neo4j into C++ objects
- Graph built; BFS/DFS explore connections
- Scores ranked via priority queue, output with reasons

Technology Stack & Methodology

Technology Stack

- Programming: C++ (OOP, STL)
- Database: Neo4j (Graph Database)
- Data Structures: Linked Lists, Graph, Queue, Priority Queue, Hash Map
- IDE: VS Code (C/C++ ext.)
- Version Control: Git / GitHub

Development Methodology

1. Requirement Analysis
2. C++/OOP Class Design
3. Graph Implementation
4. Graph Algorithms (BFS/DFS)
5. Recommendation Scoring & Testing

C++ with custom linked-list based graph structures demonstrates DSA fundamentals; Neo4j is used only as the persistence layer, keeping storage and computation separate.

Initial Progress & Team Contribution

Progress Achieved

- Requirement analysis & feature list finalized
- High-level system architecture designed
- DSA concepts and OOP class structure identified
- Graph model (nodes = users, edges = friendships) defined
- Neo4j graph schema for users, activities, friendships planned

Individual Contribution

Member	Role	Contribution
Aaradhya	Lead/Graph Dev	Custom Graph,Linked Lists , BFS/DFS and core DSA Engine
Deepak Singh Rana	DB and Back-end dev	Neo4j setup ,REST apis.auth c++engine bridge
V. Nagrey	Frountend and QA Dev	UI Dash-board,WebSockets(Chat and deployment
Akshra Raju	Recommendation Dev	Hash Ta-ble Prior Queue,Scoring

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Requirement Analysis & C++/OOP Structure
2. Phase-II: Graph Implementation & BFS/DFS Algorithms
3. Phase-III: Recommendation Engine, Neo4j Integration & Testing

Expected Outcomes

- Working prototype recommending friends & opportunities
- Ranked results based on mutual connections & shared attributes
- Demonstrated use of linked lists, graphs, hashing, priority queues
- Extensible academic prototype (non-ML, rule-based scoring)

References

1. Social Network Friend Recommendation (Scribd document)
2. Friend Recommendation Retrieval blog – recsysml.substack.com
3. Standard Data Structures & Algorithms textbook
4. Official C++ / STL / Neo4j documentation

Thank You
Questions & Suggestions